NeuroScript AI

Milestone Execution Plan

Detailed work plan for every milestone, designed so the app can be demonstrated at the end of each one

Project	NeuroScript AI - privacy-first study app for neurodivergent learners (ADHD, dyslexia, autism)
Team	Zain Tahir (23-CS-115) and Ujala Sahar (23-CS-135)
Supervisor	Mr Awais Mehmood
Department	Computer Science, University of Engineering & Technology Taxila
Course	CS-400 Final Year Project, 2026-2027
Based on	FYP proposal, Section 12 (Milestones), Section 11.1 (Distribution of Work) and the Gantt chart
Prepared	28 October 2026

Contents

1. Purpose and how to use this document 3

Roadmap at a glance . . . 3

2. The eight rules that make every milestone demonstrable 4

What is real and what is a stand-in, milestone by milestone . . . 5

3. Architecture seams, feature flags and Git workflow 6

3.1 Seams (interfaces with stand-ins) . . . 6

3.2 Feature flags (AppConfig, set with --dart-define) . . . 6

3.3 Git workflow that keeps main demoable . . . 6

3.4 What changed compared with Section 12 of the proposal . . . 7

Milestone 1 - Project Planning & Design 8

Milestone 2 - Frontend & Database Initialisation 11

Milestone 3 - Core Logic & Computer Vision 14

Milestone 4 - AI Integration & Validation 18

Part 4A - Real Gemini through the proxy . . . 18

Part 4B - Grounding-score validator . . . 19

Part 4C - On-device SLM fallback, router and queue . . . 21

Milestone 5 - Accessibility Features & QA 24

Milestone 6 - Finalisation & Documentation 27

Cross-cutting: risks and demo-day checklist 30

Project risk register . . . 30

Demo-day checklist (use at the end of every milestone) . . . 30

Appendix A - Database tables (on-device and backend) 31

Appendix B - Lesson JSON contract (freeze in M1) 31

Appendix C - Grounding score cheat sheet (proposal Section 5.1) 32

Appendix D - Suggested monorepo layout 32

Appendix E - Packages and tools by milestone 33

1. Purpose and how to use this document

The proposal splits the work into six milestones and gives each a one-line scope. This document expands every milestone into a complete work plan: tasks, step-by-step instructions, who leads, week-by-week order, deliverables, definition of done, a demo script and risks.

It also changes the **order of work inside each milestone** (not the months, not the scope) so that **at the end of every milestone the app can be shown to the supervisor or an examiner without waiting for the next milestone**. The original milestone list built the app in layers (all UI, then all database, then all vision, then all AI), which means a demo in the middle shows an app that cannot do anything useful. The plan below builds in vertical slices instead.

> **How to read each milestone**
>
> Every milestone chapter has the same layout: goal, what you can showcase, what is real vs still a stand-in, work packages (WP) with numbered steps, week plan, deliverables, definition of done (checklist), a demo script and risks. Lead names follow Table 11.1 of the proposal (Zain: backend, database, AI logic, integration; Ujala: UI/UX, QA, user testing, documentation).

Roadmap at a glance

Milestone	Months	What the app can do at the end (demo headline)	Stand-in still present	Tag
M1 Planning & Design	Oct - Nov	Clickable Figma prototype + installed app shell with profile and palette switching	All content static	v0.1.0-m1
M2 Frontend & Database	Nov - Dec	Complete offline study app: sample lessons, flashcard review with spaced repetition, concept map, encrypted DB	Lessons are bundled seed JSON	v0.2.0-m2
M3 Core Logic & Vision	Dec - Jan	Scan handwriting -> clean -> OCR -> edit -> cloud proxy -> lesson -> review, on a real phone	Server uses a labelled mock generator	v0.3.0-m3
M4 AI & Validation	Jan - Feb	4A real Gemini lessons; 4B grounding-score badges; 4C airplane-mode fallback and queue	None after 4C (each part works alone)	v0.4.1-4a / 4b / 4c
M5 Accessibility & QA	Feb - Mar	Three profile experiences, read-aloud, simplify mode, QA and evaluation report	Comprehension check is optional	v0.5.0-m5
M6 Finalisation	Mar - Apr	Release build, UAT with SUS, final report, poster, presentation	None	v1.0.0

2. The eight rules that make every milestone demonstrable

These rules apply to the whole project. If a task breaks one of them, change the task, not the rule.

1. **Vertical slices, not layers.** Every milestone ends with a working app that a person can hold in their hand and use. We never finish 'all the UI' or 'all the backend' alone. Each milestone adds one thin, complete slice from screen to storage.
2. **Build seams first, fill them later.** Each big component (lesson generator, OCR, validator, TTS, fallback) sits behind a Dart interface with a working stand-in (seed data, mock, or stub). Later milestones swap the stand-in for the real thing without touching the screens.
3. **One data contract from day one.** The Lesson JSON (Appendix B) is fixed in Milestone 1. Seed data, the mock generator, Gemini and the on-device model all produce the same shape, so the UI never changes when the AI changes.
4. `main` **is always demoable.** Work happens on feature branches and merges only when CI is green. The `main` branch must build an installable APK at every moment, not only at milestone end.
5. **Feature flags decide what the demo shows.** A single `AppConfig` (set with `--dart-define`) switches generator mode, capture, grounding, fallback, simplify and demo mode. A half-finished feature is hidden, never shown broken.
6. **Honest labels.** Anything that is a stand-in is labelled on screen (for example a small 'Demo generator' tag) and listed in that milestone's limitations slide. We never present a mock as the real AI.
7. **Every milestone has a safety net.** A tagged release with a signed APK, a 90-second screen recording, seeded demo data, and a rehearsed demo script. If Wi-Fi, the server or the API fails on demo day, the recording and the offline path still work.
8. **Small, early usability checks.** Two 15-minute guerrilla tests with 2-3 peers at the end of Milestone 2 and Milestone 4 catch design problems long before formal UAT in Milestone 6.

What is real and what is a stand-in, milestone by milestone

Read each column as the state of the app at the **end** of that milestone. Amber cells are stand-ins that are clearly labelled in the app. No column needs a cell from a later column to work.

Component	M1	M2	M3	M4A	M4B	M4C	M5	M6
App shell + navigation	Real	Real	Real	Real	Real	Real	Real	Final
Profile themes (ADHD / Dyslexia / Autism)	Palette stub	Basic	Basic	Basic	Basic	Basic	Full	Final
Encrypted local DB (drift + SQLCipher)	-	Real	Real	Real	Real	+ queue	Real	Final
Lesson content source	-	Seed JSON	Mock gen.	Gemini	Gemini	Gemini + local	+ Simplify	Final
Lesson viewer, cards, concept map	Mock-up	Real	Real	Real	+ badges	+ badges	Adaptive	Final
Spaced-repetition review	-	Real	Real	Real	Real	Real	Real	Final
Camera + OpenCV cleanup	-	-	Real	Real	Real	Real	Tuned	Final
ML Kit OCR + review screen	-	-	Real	Real	Real	Real	Tuned	Final
ASP.NET Core proxy	-	-	Real (mock gen.)	Real	Real	Real	+ /simplify	Deployed
Grounding-score validator	-	-	Schema only	Schema only	Real	Real	Calibrated	Final
On-device SLM fallback + queue	-	-	-	-	-	Real	Tuned	Final
Text-to-Speech read-aloud	-	-	-	-	-	-	Real	Final
Evidence for the report	Req. + designs	Guerrilla test 1	OCR baseline	Latency log	Validator metrics	Device perf.	QA report	UAT + SUS

Colours: grey = not built yet, amber = stand-in or partial, green = real, dark green = final/release quality.

3. Architecture seams, feature flags and Git workflow

These three things are what make the independent milestones possible. Build them in Milestone 1 and M2, and every later milestone becomes a replacement, not a rewrite.

3.1 Seams (interfaces with stand-ins)

Seam (Dart / C# interface)	Stand-in -> real implementation by milestone
`LessonGenerator.generate(note, profile)`	`SeedLessonGenerator` (M2) -> `RemoteLessonGenerator` + server `MockLessonGenerator` (M3) -> `GeminiLessonGenerator` (4A) -> `GenerationRouter` cloud / local / queue (4C)
`OcrService.read(images)`	`FakeOcrService` with fixed text for tests -> `MlKitOcrService` (M3)
`GroundingValidator.check(item, lines)`	`SchemaOnlyValidator` (M3) -> lexical + entity validator v1 (4B week 3) -> full validator v2 with MiniLM (4B)
`LocalLlm.generate(prompt)`	`DisabledLocalLlm` -> `LlamadartLocalLlm` (4C)
`TtsService`	`NoopTtsService` -> `FlutterTtsService` (M5)
`Clock`, `Connectivity`, `DeviceCapabilities`	Fakes in tests (simulate midnight, offline, low RAM); real implementations in the app
Server `ILessonGenerator`	`MockLessonGenerator` (M3) -> `GeminiLessonGenerator` (4A), selected with `Generator:Mode`

3.2 Feature flags (`AppConfig`, set with `--dart-define`)

Flag	Values	Default by milestone
`generatorMode`	seed | remote	M1-M2 seed; M3 onward remote
`enableCapture`	true | false	false in M1-M2; true from M3
`groundingMode`	off | schema | lexical | full	M3-4A schema; 4B lexical then full
`enableLocalFallback`	true | false	false until 4C
`enableSimplify`, `enableTts`	true | false	false until M5
`demoMode`	true | false	true for demos: shows 'Demo generator' tag and the debug menu; false in the release build

3.3 Git workflow that keeps `main` demoable

- Branches: `main` (always installable), short-lived `feature/<wp-id>-name` branches, optional `release/mN` branch during the last week of a milestone.
- Pull requests must pass CI (`flutter analyze`, `flutter test`, `dotnet test`) and be reviewed by the other teammate before merge.
- Milestone close-out: merge, tag (for example `v0.2.0-m2`), build a signed APK, attach it to a GitHub Release with release notes and the limitations list.
- Never commit keys, `.env` files or signing keystores. Server secrets live only in the host's environment variables; the app only knows the proxy URL and a build key.
- Rule of thumb for scope: if a task cannot be merged behind a flag by the milestone's last week, cut or postpone it instead of showing it broken.

3.4 What changed compared with Section 12 of the proposal

The six milestone names, the months and the Gantt chart stay exactly as submitted. Only the internal order of tasks changes:

Change	Problem in the original order	How this plan fixes it
Flutter shell + CI built in M1	Proposal had the Flutter project start in M2, so M1 ended with documents only.	M1 now ends with a runnable APK next to the Figma prototype, so even the first milestone can be shown as an app.
Seed lessons in M2	UI and DB alone would have shown empty screens until OCR and AI existed.	A bundled seed lesson (Lesson JSON contract) fills the app, so M2 is a complete offline study app.
Mock generator in M3	OCR + backend alone would have ended in text with nothing useful to do with it.	The ASP.NET proxy ships with a labelled mock generator, so scan -> OCR -> server -> lesson -> review works end to end before Gemini.
M4 split into 4A / 4B / 4C	Gemini, fallback and validation were one block and each depended on the others.	4A real Gemini, 4B validator (works on mock or Gemini output), 4C fallback. Any one can slip without breaking the others.
Validator before fallback	Proposal order was fallback first, validation last.	Validator first means every local-model answer is already checked when the fallback arrives.
Basic themes in M2, full in M5	Themes and TTS arrived only in M5, so M2-M4 demos had one generic look.	Three basic profile palettes exist from M2; M5 adds fonts, chunking, simplify mode and TTS.
Guerrilla tests at end of M2 and M4	First user test was in M6, too late to change design.	Short peer tests earlier; formal UAT with SUS stays in M6 as the proposal says.

Milestone 1 - Project Planning & Design

When	October - November (about 5 weeks)
Release tag	v0.1.0-m1

Goal. Lock the problem, requirements, design and architecture, and finish with a runnable app shell, so the supervisor sees an installed app and a clickable prototype instead of only documents.

What you can showcase at the end of this milestone

A clickable high-fidelity Figma prototype of every planned screen, plus a real APK on a phone that launches, shows onboarding, lets you pick a profile (ADHD / Dyslexia / Autism) and switches between three colour palettes. Architecture, ER diagrams and the API contract are on screen.

Real at the end	Still a stand-in
Figma prototype, requirements, diagrams, GitHub monorepo + CI, Flutter app shell (navigation, theme switcher).	All content is static placeholders. No database, no camera, no AI.

Work packages

WP1.1 Literature survey and requirements

Lead: Ujala (lead), Zain (assist)

1. Collect and summarise the 15 references already cited in the proposal plus any new ones; keep a shared reference sheet (title, key finding, how it affects our design).
2. Finish the competitor matrix (Quizlet, Knowt, Anki, ChatGPT, Speechify) and write one paragraph per product on what it does badly for neurodivergent students.
3. Run a short survey or 5-minute chats with 8-10 students (include neurodivergent students if reachable) about how they take and revise notes.
4. Write three personas (ADHD, dyslexia, autism) with goals, pain points and what 'success' looks like.
5. Turn findings into a numbered requirement list: functional (FR-01...), non-functional (NFR-01...), each marked Must / Should / Could.

Output: Requirements table with IDs and priorities; three persona sheets; reference sheet.

WP1.2 Proposal finalisation

Lead: Ujala (lead)

1. Apply supervisor feedback, regenerate the table of contents and figure numbers, check every citation in the reference list.
2. Attach the Turnitin and AI-report appendices in the department's format and collect the supervisor's signature.
3. Freeze the proposal as version 1.0 in the repository (`/docs/proposal`).

Output: Signed proposal PDF/DOCX in the repo.

WP1.3 Wireframes to high-fidelity prototype

Lead: Ujala (lead)

1. List the screens: Splash/Onboarding, Profile select, Library, Capture, OCR review, Generating/status, Lesson overview, Flashcard review, Concept map, Progress, Settings, plus Offline, Error and Empty states.
2. Draw low-fidelity wireframes first and review them with Zain for feasibility (can Flutter build this in one week?).
3. Define design tokens: colour palettes per profile, type scale, spacing, corner radius, elevation, icon set.

4 Build the clickable Figma prototype for the main journey: onboarding -> profile -> scan -> review text -> lesson -> flashcards -> progress.

5 Check every palette for WCAG contrast (4.5:1 for body text), 48 dp minimum touch targets, and no information carried by colour alone.

Output: Figma file with tokens and a clickable end-to-end prototype; exported PNGs for the report.

WP1.4 Architecture, diagrams and API contract

Lead: Zain (lead), Ujala (assist)

1 Finalise the working flow diagram (Figure 5.1) and a component diagram (App, Proxy, Gemini, on-device SLM, local DB).

2 Draw two sequence diagrams: (1) scan -> OCR -> proxy -> validate -> save; (2) cloud failure -> on-device SLM -> queue.

3 Finish both ER diagrams (on-device Figure 5.2, backend Figure 5.3) and mark which tables are extensions of the proposal (Appendix A).

4 Write the proxy API contract as an OpenAPI draft: `GET /health`, `POST /api/v1/lessons/generate` (later `POST /api/v1/text/simplify`), error codes and rate-limit headers.

5 Freeze the Lesson JSON schema (Appendix B) and commit it to `/docs/contracts/lesson.schema.json`.

Output: Diagram set, OpenAPI draft, Lesson JSON schema (all in `/docs`).

WP1.5 Repository, tooling and app shell

Lead: Zain (lead)

1 Create one GitHub monorepo: `/app` (Flutter), `/backend` (ASP.NET Core), `/docs`, `/design`. Protect `main`, require PRs and green CI.

2 Create the GitHub Project board with one milestone per project milestone and issues for every work package here.

3 Add GitHub Actions: `flutter analyze` + `flutter test` for `/app`, `dotnet build` + `dotnet test` for `/backend`. Add `flutter_lints`, `.editorconfig`, PR template, `.gitignore` that blocks keys and `.env` files.

4 If a Flutter structure already exists, audit it against Appendix D and fix the folder layout now (feature-first folders, Riverpod, go_router).

5 Create the `AppConfig` class with all feature flags (Section 5) and the theme system with three placeholder palettes.

6 Build a release APK, install it on a physical phone, tag `v0.1.0-m1` and attach the APK to a GitHub Release.

Output: Green CI, installable APK, tagged release.

Week-by-week plan

When	Focus
Week 1	Requirements, survey, repository, CI
Week 2	Wireframes, architecture and sequence diagrams
Week 3	High-fidelity prototype, ER diagrams, OpenAPI draft, Lesson schema
Week 4	App shell, theme switcher, supervisor review of design
Week 5	Fix review comments, freeze, tag, rehearse demo

Deliverables

- Signed proposal v1.0
- Requirements + personas
- Figma prototype

- Diagrams, ER, OpenAPI, Lesson schema
- GitHub repo + CI + board
- Installable APK `v0.1.0-m1`
- Monthly report (October)

Definition of done

☐ Supervisor has seen the prototype and approved the design direction
☐ Every requirement has an ID and priority
☐ Both ER diagrams and both sequence diagrams are consistent with Appendix A and B
☐ CI is green on `main`; APK installs and launches on a real phone
☐ Tag `v0.1.0-m1` exists with the APK attached

Demo script (about 3-5 minutes)

1. Open with the 30-second problem statement (messy handwritten notes, hallucinating AI, privacy).
2. Open the Figma prototype on a laptop and click through the whole journey.
3. On a phone, launch the APK: splash -> onboarding -> choose a profile -> see the palette change -> open Settings.
4. Show the architecture diagram and the GitHub board with milestones.
5. Close with the roadmap slide: what you will be able to show next month (M2).

> **Say this honestly in the demo (limitations)**
> No real data, no storage, no camera, no AI. Say so up front.

Risks and fallbacks

Risk	Mitigation / fallback
Wireframes drag on	Time-box to 5 working days; hi-fi only for the main journey, other screens stay wireframes until M2.
Requirements too large	Mark Could-have items and move them to the 'Future work' list in the report.

Milestone 2 - Frontend & Database Initialisation

When	November - December (about 6 weeks)
Release tag	v0.2.0-m2

Goal. Deliver a complete offline study app that works with bundled sample lessons. Everything the student does after a lesson exists (review, progress, concept map) is real; only the way a lesson is created is still stood in for by seed data.

What you can showcase at the end of this milestone

Pick a profile, browse a library of sample lessons, open one, review its flashcards with spaced repetition, explore the concept map, tap 'Where is this from?' to see the highlighted source lines of the original note. Close the app, reopen it, and progress is still there. Pull the database file off the phone and show it is unreadable (encrypted).

Real at the end	Still a stand-in
All screens except capture, drift database encrypted with SQLCipher, repositories, spaced-repetition scheduling, progress screen, basic profile palettes.	Lessons come from `assets/seed/*.json`, written in the final Lesson JSON contract with realistic source lines. The Capture button is hidden by `enableCapture=false`. In debug builds only, a developer 'paste text' box imports test notes.

Work packages

WP2.1 Local database layer

Lead: Zain (lead)

1. Add `drift`, `drift_dev`, `build_runner`, `sqlcipher_flutter_libs`, `flutter_secure_storage`, `crypto`. Remove `sqlite3_flutter_libs` (it conflicts with SQLCipher).
2. Define tables exactly as in Appendix A: UserProfiles, SourceNotes, Lessons, Flashcards, ReviewProgress. Use foreign keys with cascade delete so removing a note removes its lessons, cards and progress.
3. Generate a random 256-bit key with `Random.secure()` on first launch, store it in `flutter_secure_storage`, and open the database with `PRAGMA key` in the connection setup.
4. Set `schemaVersion = 1`, write the migration scaffold and a migration test now, so later changes (queue table in M4) are safe.
5. Implement DAOs and repository interfaces (`NoteRepository`, `LessonRepository`, `ReviewRepository`, `ProfileRepository`) so UI code never touches drift directly. Provide in-memory implementations for tests.
6. Compute `DocumentHash` as SHA-256 of the normalised note text; used later for on-device caching and duplicate detection.
7. Write a first-run seed importer that loads `assets/seed/*.json` inside one transaction and is idempotent.
8. Prove encryption: pull the `.db` file with `adb`, try to open it with the plain `sqlite3` tool (it must fail), then open it with the key (it must work). Screenshot this for the report.

Output: Encrypted database with repositories, seed importer and passing DAO tests.

WP2.2 Spaced repetition and progress logic

Lead: Zain (lead)

1. Implement a Leitner-style scheduler: intervals 1, 3, 7, 14, 30 days; a correct answer moves the card up one box, a wrong answer returns it to 1 day. This needs only the existing `SpacedRepetitionInterval` and `NextReviewDate` columns.

2. Write the daily queue builder: due cards first, then new cards, capped by a session size that depends on the profile (default 10, ADHD profile 5).
3. Add progress statistics: cards due today, streak days, cards mastered, per-lesson completion.
4. Inject a clock interface so tests can move time forward; test day boundaries and time-zone changes.

Output: Pure-Dart scheduler with a unit-test table of at least 15 scenarios.

WP2.3 State management and navigation

Lead: Zain (lead), Ujala (assist)

1. Create Riverpod providers for repositories and `AsyncNotifier` controllers per feature; override repositories in tests.
2. Set up `go_router` with named routes and guards (no profile -> onboarding).
3. Define one error pattern (typed failures + `AsyncValue.error` -> friendly screen with a Retry button) and use it everywhere.
4. Keep all strings in one place so wording can be simplified per profile later (M5).

Output: Navigation map matching the prototype and tested providers.

WP2.4 Screens

Lead: Ujala (lead)

1. Onboarding and profile selection: three clear cards, one sentence each, saved to `UserProfiles`.
2. Library: lesson list with title, card count, due count; empty state with a friendly illustration; Capture entry present but hidden by flag.
3. Lesson overview: title, short summary, buttons 'Review now' and 'Concept map'.
4. Flashcard review: flip card, 'Got it' / 'Not yet', progress bar, end-of-session summary.
5. Concept map: nodes and labelled edges drawn with `graphview` or a `CustomPainter`, inside `InteractiveViewer` for zoom/pan; tapping a node opens a bottom sheet with its source line(s).
6. 'Where is this from?' sheet on every card and node: shows the original note lines and highlights the cited `SourceLineReference` lines.
7. Progress screen and Settings (switch profile, palette, text size, delete all data with confirmation).

Output: All screens built from the Figma file, working against real repositories.

WP2.5 Design system implementation

Lead: Ujala (lead)

1. Build `ThemeData` factories for the three profiles from the Figma tokens; one source of truth for colour, type and spacing.
2. Create reusable widgets (`AppButton`, `CardTile`, `ProgressRing`, `SectionHeader`) so screens stay consistent.
3. Test text scaling from 1.0x to 2.0x with no overflow, add `Semantics` labels, and verify touch targets of at least 48 dp.

Output: Theme + widget kit with a small widget gallery screen (debug only).

WP2.6 Tests and the M2 gate

Lead: Zain and Ujala

1. Unit tests: scheduler, repositories, hash, seed importer. Widget tests: onboarding, review flow, empty state.
2. Run on two physical phones (one low-end) and one tablet or large emulator; fix layout bugs.
3. Guerrilla test #1: 2-3 peers use the app for 15 minutes with no help; record where they hesitate and fix the top three issues.

Output: Green CI, test notes, fixed top issues.

Week-by-week plan

When	Focus
Week 1	Database schema, encryption spike, seed format
Week 2	Repositories, seed importer, scheduler, providers, routing
Week 3-4	Screens (library, lesson, review, concept map, progress, settings)
Week 5	Themes, accessibility checks, tests, guerrilla test
Week 6	Buffer, bug fixing, demo rehearsal, tag `v0.2.0-m2`

Deliverables

- APK `v0.2.0-m2`
- Encrypted DB proof screenshots
- Seed lessons (at least 3 different subjects, one per profile style)
- Unit + widget tests
- Guerrilla test notes
- Monthly reports (November, December) and mid-term progress report

Definition of done

- ☐ App works fully offline in airplane mode
- ☐ Progress survives force-close and phone restart
- ☐ Database file is unreadable without the key
- ☐ Every card has a working 'Where is this from?' sheet
- ☐ Text scaled to 2.0x shows no overflow on any screen
- ☐ Tests green; tag `v0.2.0-m2` with APK attached

Demo script (about 3-5 minutes)

1. Fresh install -> onboarding -> choose 'Dyslexia' -> see the palette.
2. Open the library -> open the seed lesson on photosynthesis (or any subject).
3. Review 5 cards, answer some wrong, show the summary and the next review dates.
4. Open the concept map, pinch to zoom, tap a node, open 'Where is this from?'.
5. Force-close and reopen: progress is still there. Show the encrypted DB file on a laptop.
6. Switch profile to ADHD: session size drops to 5 cards.

> **Say this honestly in the demo (limitations)**
>
> Lessons are bundled samples; you cannot scan your own notes yet. Say: 'The creation path arrives in Milestone 3; everything after it is already real.'

Risks and fallbacks

Risk	Mitigation / fallback
SQLCipher setup fails on some devices	Test on 2 devices in week 1. Keep an in-memory/unencrypted debug flag only for tests, never for release.
Concept map library is awkward	Fall back to a simple tree/list layout using `CustomPainter`; the JSON stays the same.

Milestone 3 - Core Logic & Computer Vision

When	December - January (about 7 weeks plus buffer)
Release tag	v0.3.0-m3

Goal. Make the full pipeline real from camera to lesson on a real phone and a deployed server. Gemini is not needed yet: the proxy returns lessons from a clearly labelled mock generator that follows the same contract.

What you can showcase at the end of this milestone

Photograph a real handwritten page -> see the cleaned image -> read the OCR text line by line and fix a mistake -> tap Generate -> the phone calls our deployed ASP.NET server -> a lesson appears in the library (tagged 'Demo generator') -> review it with spaced repetition. Also show the server health page, the rate limit rejecting a flood, and the OCR accuracy numbers.

Real at the end	Still a stand-in
Camera, OpenCV preprocessing, ML Kit OCR, OCR review/edit screen, SourceNotes saved, ASP.NET Core proxy (API key, validation, rate limiting, logging, health), API client with timeouts and error states, cloud deployment, OCR test set and metrics.	`MockLessonGenerator` on the server builds simple extractive cards (patterns like 'X is Y', 'X: Y', numbered facts) with correct `sourceLines`. It exists only to exercise the pipeline and is switched off in release builds. Validation is schema-only (grounding score arrives in M4B). There is no Gemini and no fallback yet.

Work packages

WP3.1 Capture screens

Lead: Ujala (UI lead), Zain (logic)

1. Permission flow with `permission_handler` and a friendly explanation screen for denied or permanently denied cases.
2. Camera screen with `camera`: live preview, guide frame, flash toggle, tap to focus, shutter, retake.
3. Crop and rotate screen after capture; allow 1-5 pages per note (pages are joined with page markers).
4. Add a 'tips for good photos' card (flat page, even light, dark pen, no shadow); show it before the first scan.
5. Gallery import with `image_picker` behind a debug-only flag, used for repeatable testing with a fixed set of photos.

Output: Capture flow that returns one or more image files to the pipeline.

WP3.2 Image preprocessing with OpenCV

Lead: Zain (lead)

1. Add `opencv_dart` (or the lighter Dart `image` package if OpenCV build size is a problem) and run all processing in a background isolate so the UI stays smooth.
2. Implement the pipeline: resize to about 2000 px on the long side -> grayscale -> contrast normalisation (CLAHE) -> light denoise -> adaptive threshold -> deskew. Each step is a separate function with a toggle.
3. Run an A/B test on your own photos: raw vs each step on or off, measured by character error rate (CER). Keep only the steps that lower CER.
4. Delete all temporary images after OCR finishes. Images are never stored unless the user explicitly saves them (privacy promise).

Output: Preprocessing module with a short table of which steps helped and by how much.

WP3.3 OCR and the review screen

Lead: Zain (lead), Ujala (UI)

1. Integrate `google_mlkit_text_recognition` (Latin script). Convert blocks -> lines in reading order, fix hyphenated line breaks, and number the lines from 1. These numbers are what `sourceLines` refer to everywhere later.
2. Build the OCR review screen: numbered lines, tap to edit, add or delete a line, merge two lines, and a clear 'Looks right' button. Users correct the text before anything is sent anywhere, so a weak OCR result never blocks a demo.
3. Save the confirmed text into `SourceNotes` (`RawOCRText`, `DocumentHash`, `CreatedAt`). If the hash already exists, offer to open the existing lesson instead of generating again.
4. Decision gate at the end of week 1: measure median CER on 10 photos. If it is above 25 percent even after preprocessing, switch to the contingency in the risk list below.

Output: Capture -> clean -> OCR -> edit -> saved note, working on a physical phone.

WP3.4 ASP.NET Core proxy

Lead: Zain (lead)

1. Create the solution with projects `Api` (minimal API), `Core` (contracts and interfaces), `Infrastructure` (generators, SQLite via EF Core) and `Tests` (xUnit). Use the current LTS version of .NET.
2. Endpoints: `GET /health` and `POST /api/v1/lessons/generate` with a body of `{ noteHash, lines[], profile, mode, clientVersion }`. Response is the Lesson JSON; errors use `ProblemDetails` with stable codes (`RATE_LIMITED`, `INVALID_INPUT`, `UPSTREAM_ERROR`, `TIMEOUT`).
3. Security: `X-Api-Key` header per build plus a random install ID; HTTPS only; request size limits (maximum lines and characters); built-in `AddRateLimiter` per install ID returning 429 with `Retry-After`; keys and settings only from environment variables or user-secrets.
4. Privacy by design: the server is stateless for content. It does not store note text or generated lessons. The SQLite database holds only metadata (RequestLog: install hash, endpoint, status, latency, provider, timestamp). This matches the backend ER (Figure 5.3).
5. Define `ILessonGenerator` and implement `MockLessonGenerator` first; the Gemini implementation arrives in M4A without changing the endpoint.
6. Structured logging with Serilog (log hashes and sizes, never note content); Swagger UI in development only.
7. xUnit tests: input validation, rate limiter, error mapping, endpoint tests with `WebApplicationFactory`.

Output: Backend running locally and passing tests; OpenAPI document matches the M1 draft.

WP3.5 Client integration

Lead: Zain (lead)

1. Define `LessonGenerator` (Dart interface) with `SeedLessonGenerator` (M2) and `RemoteLessonGenerator` (this milestone); chosen by `AppConfig.generatorMode`.
2. Use `dio` with a 5 s connect timeout, 30 s receive timeout and retry with backoff on 5xx only (never on 4xx).
3. Parse the response with strict schema validation; on mismatch show a typed error, never a crash or a half-built lesson.
4. Pipeline status screen: Capturing -> Cleaning -> Reading -> Generating -> Saved, with a Cancel button and clear error screens for no internet, timeout, server error and rate limit.
5. Save the lesson, cards and `SourceLineReference` values in a single database transaction.

Output: End-to-end flow from camera to saved lesson.

WP3.6 Deployment

Lead: Zain (lead)

1. Deploy the proxy to a free or student-tier cloud host (the proposal suggests Azure). Configure environment variables, HTTPS and a health check.
2. Add a GitHub Actions deploy job and a smoke-test script that calls `/health` and a sample generate request.
3. Use build flavours via `--dart-define`: `dev` points to a laptop on the same Wi-Fi, `prod` points to the cloud URL.
4. Free tiers may cold-start; add a 'warm-up' call to the demo checklist (open `/health` two minutes before presenting).

Output: Public HTTPS URL, working from a phone on mobile data.

WP3.7 Test data and OCR evaluation

Lead: Ujala (lead), Zain (assist)

1. Collect at least 30 photos of handwritten notes from at least 5 different writers: ruled and plain paper, pen and pencil, good and poor light, neat and messy.
2. Type the ground-truth text for each photo and store both in `/docs/testdata/ocr`.
3. Write a small script that computes CER and WER (Levenshtein distance) for raw vs preprocessed images and outputs a table.
4. Keep these same photos as the shared dataset: in M4B they become the source for the grounding-score evaluation, so label them carefully now.

Output: OCR baseline report (CER and WER by condition) and a reusable dataset.

Week-by-week plan

When	Focus
Week 1	Camera, permissions, first OCR on device, go/no-go on OCR quality
Week 2	Preprocessing, review screen, saved notes
Week 3	Backend skeleton, mock generator, security, rate limit, tests
Week 4	Client integration end to end on a phone against local backend
Week 5	Cloud deployment, error states, smoke tests
Week 6	OCR dataset, metrics, bug fixing, guerrilla notes
Week 7	Buffer, demo rehearsal, tag `v0.3.0-m3`

Deliverables

- APK `v0.3.0-m3` and public proxy URL
- OCR baseline report + dataset
- Backend test suite
- Sequence diagram updated to match the real flow
- Monthly report (January)

Definition of done

- ☐ A new handwritten page goes from camera to saved lesson in under about 15 seconds on mobile data
- ☐ OCR text is always editable before generation
- ☐ Rate limiter returns 429 and the app shows a clear message
- ☐ No temporary images remain after a scan; server stores no note content
- ☐ Mock generator output is labelled 'Demo generator' in the UI
- ☐ Backend tests and app tests green; tag with APK attached

Demo script (about 3-5 minutes)

1. Warm up the server, then start the app in airplane mode off.
2. Scan a page of your own handwriting; show the cleaned image.
3. On the review screen fix one wrong word and press Generate.
4. Watch the status steps; open the new lesson and review two cards; show the 'Where is this from?' sheet pointing to the exact lines.
5. Show the server health page and the OCR accuracy table.
6. Mention honestly: AI content is a mock now; Gemini arrives in the next milestone.

Say this honestly in the demo (limitations)

Lesson quality is basic because it is rule-based. This path is a scaffold for the pipeline and not a fallback engine: the proposal's fallback is the on-device SLM in M4C.

Risks and fallbacks

Risk	Mitigation / fallback
ML Kit struggles with messy handwriting	Guided capture, preprocessing, and the editable review screen. Contingency decided at the week-1 gate: add an optional OCR step through the proxy using a Gemini vision call, keeping the same `OcrResult` shape. Present it honestly as a measured trade-off in the report.
Free-tier cloud cold starts or downtime	Warm-up in the demo checklist; a dev flavour that points to a laptop hotspot as backup.
OpenCV adds too much APK size	Use ABI splits, or switch to the Dart `image` package for the few steps that matter.

Milestone 4 - AI Integration & Validation

When	January - February (about 8 weeks, three independently shippable parts)
Release tag	v0.4.0-m4 (sub-tags v0.4.1-4a, v0.4.2-4b, v0.4.3-4c)

Goal. Replace the mock with real AI, then make that AI trustworthy, then make it resilient. The three parts are ordered so each one is demoable on its own and none needs the next one to be finished.

What you can showcase at the end of this milestone

See the three sub-demos below. The overall story at the end: scan a note, receive a real AI lesson where every card carries a verified badge; corrupt one card on purpose and watch the validator remove it; switch on airplane mode or simulate a rate limit and the lesson is still produced on the device.

Structure. Milestone 4 is delivered as three parts. Each part has its own demo, tag and definition of done. The order (4A -> 4B -> 4C) is the order of dependency in value, but none of the later parts is needed to show an earlier one, and 4B works on mock output so it does not even need 4A.

Part 4A - Real Gemini through the proxy

Timing: Weeks 1-2 (tag v0.4.1-4a)

Showcase for 4A

The exact same scan flow now returns genuine AI lessons built from your own notes. Different profiles give visibly different wording (short chunks for ADHD, plain short sentences for dyslexia, literal language for autism). Scanning the same note twice is instant because of the on-device cache.

Real at the end	Still a stand-in
`GeminiLessonGenerator`, prompt files for three profiles, structured JSON output, retry and error mapping, on-device cache by `DocumentHash`.	Grounding is still schema-only; items are marked 'Not yet verified' internally. No local fallback. `Generator:Mode` can still be set to `Mock` as a demo-day backup if the API is down.

WP4A.1 Prompt design (closed-book compiler)

Lead: Zain (lead)

1. Write system instructions: use only the numbered lines provided, never add outside facts, cite `sourceLines` for every card and node, return fewer items instead of inventing, and follow the Lesson JSON schema exactly.
2. Treat the note as data: wrap the lines in clear delimiters and instruct the model to ignore any instructions that appear inside the notes (prompt-injection defence).
3. Write profile style blocks: ADHD (single idea per card, short answers, small batches), dyslexia (short plain sentences, no jargon unless in the note), autism (literal wording, no idioms or metaphors).
4. Set low temperature (0 to 0.2), cap the number of items, and request JSON output with a response schema.
5. Store prompts as versioned files in `/backend/prompts/v1/` and include the prompt version in logs and in the response metadata.
6. Optional extension from your earlier idea: add an optional `verificationQuestion` field (a question whose answer is in the note) to support cross-questioning. Keep it optional so the proposal scope is not exceeded.

Output: Prompt pack v1 with examples of good and bad outputs.

WP4A.2 Gemini client in the proxy

Lead: Zain (lead)

1. Use a typed `HttpClient` with `Microsoft.Extensions.Http.Resilience` for timeout (about 20 s), retry with jitter and a circuit breaker.
2. Keep the model name in configuration (for example `Gemini:Model`), never hard-coded, because model names and free-tier limits change. Check the current limits and data-use terms at implementation time.
3. Map upstream failures: 429 -> `RATE_LIMITED` with `Retry-After`; 5xx or timeout -> `UPSTREAM_ERROR`; safety-blocked response -> `CONTENT_BLOCKED` with a friendly message.
4. Parse and validate the model's JSON against the schema; if invalid, retry once with a short repair instruction, otherwise return an error.
5. Register the generator with dependency injection and select it with `Generator:Mode = Gemini | Mock`.

Output: Real lessons through the same endpoint; tests with a fake Gemini handler.

WP4A.3 Client cache and UX

Lead: Zain (lead), Ujala (UI)

1. Look up `(DocumentHash, profile, promptVersion)` in the local database before calling the server; if found, open the lesson instantly.
2. Show generation progress, a friendly rate-limit message, and keep the review screen unchanged.
3. Run 20 notes from the M3 dataset through Gemini and log latency and obvious problems.

Output: Cache hit path and a latency table.

Definition of done for part 4A

- ☐ Real lesson from a new note in under about 15 seconds on mobile data
- ☐ Repeat scan opens from cache instantly
- ☐ Invalid or blocked responses never crash the app
- ☐ Gemini key exists only on the server (check the APK with a string search)

Demo script for part 4A

1. Scan a note and generate: show the real cards.
2. Switch the profile and regenerate to show different styles.
3. Scan the same note again: instant open.
4. Open the server log and show that only metadata, not note content, is logged.

Part 4B - Grounding-score validator

Timing: Weeks 3-5 (tag v0.4.2-4b)

Showcase for 4B

Every card and concept-map node shows a small 'Verified' badge with the source lines it came from. Change a number inside a generated card in debug mode and show the validator reject it. A 'Why was this removed?' debug panel lists rejected items with their scores. It works identically on mock output or Gemini output.

Real at the end	Still a stand-in
On-device validator implementing Section 5.1 of the proposal: normalisation, n-gram containment L, MiniLM semantic score Sem, combined score G, entity check X, accept/regenerate/discard rule, metrics and calibration.	v1 (week 3) is lexical plus entity check only; v2 (weeks 4-5) adds the MiniLM semantic score. v1 is already demoable, so a delay in the embedding model never blocks the milestone.

WP4B.1 Text normalisation and evidence window

Lead: Zain (lead)

1. Implement `normalize()`: lowercase, strip punctuation, repair OCR hyphenation, split into lines, remove stop words (keep a small stop-word list file).

2 For each item, build evidence E = cited lines plus one neighbouring line left and right.

3 Write unit tests with small hand-made notes.

Output: Normaliser and evidence builder with tests.

WP4B.2 Lexical score and entity check (v1)

Lead: Zain (lead)

1 Compute unigram and bigram containment: `C1`, `C2`, and `L = (C1 + C2) / 2`.

2 Entity check X: extract numbers, dates, formulas and proper names from the item; each must appear exactly in E, otherwise X = 0 and G = 0.

3 Apply the decision rule with the starting thresholds (accept at G >= 0.75 and X = 1; regenerate once between 0.50 and 0.75; discard below 0.50). In v1, use L as G.

4 Add a `strict:true` request flag so the proxy can run the second 'quote-only' prompt for the single regeneration.

5 Save only accepted items; if every item is rejected show 'We could not verify enough. Rescan or edit the text.'

Output: Working validator v1, badge UI and the debug 'Why removed?' panel.

WP4B.3 Semantic score with on-device MiniLM (v2)

Lead: Zain (lead)

1 Choose a quantised MiniLM-class sentence-embedding model (about 22 million parameters) and run it through an ONNX Runtime or TFLite Flutter binding (verify the current maintained package when you start).

2 Port the WordPiece tokenizer to Dart using the model's vocabulary file and check its tokens against a Python reference on 20 sentences.

3 Run embedding in a background isolate, load lazily, and cache embeddings per OCR line so each note is embedded once.

4 Compute `Sem = max_j cos(v_g, v_j)` and `G = 0.6 * Sem + 0.4 * L`, with weights read from `grounding_config.json`.

5 Measure latency and memory on a 4 GB phone; target well under a second per note after the model is loaded.

Output: Validator v2 with the full proposal formula.

WP4B.4 Evaluation set and calibration

Lead: Zain (lead), Ujala (labelling)

1 Generate lessons for about 30 notes; label each item Supported or Unsupported by hand (aim for about 300 items). Add deliberately corrupted items (wrong number, swapped name, invented fact) so the set contains enough negatives.

2 Run a grid search over alpha and the two thresholds to maximise the F1 score of hallucination detection; write the chosen values into `grounding_config.json`.

3 Report Grounded Rate (accepted / generated) and residual Hallucination Rate (unsupported among accepted) for Gemini output, and later for the local SLM.

4 Run the same notes through a commercial chat assistant with a plain prompt and label its flashcards the same way for the comparison the proposal promises. State clearly that the validator checks consistency with the student's notes, not truth in the world.

Output: Calibrated thresholds and the first evaluation table for the report.

Definition of done for part 4B

☐ Wrong number, swapped name and invented fact are all caught in the unit-test table

☐ Paraphrased but correct cards are accepted

☐ Validator runs fully offline

☐ Each shown item has non-empty source lines

☐ Metrics script reproduces the evaluation table

Demo script for part 4B

1. Generate a lesson and open a card to show the Verified badge and highlighted source lines.
2. In the debug panel edit a card to change a date and show it rejected with X = 0.
3. Show the 'Why removed?' list with scores for a rejected paraphrase.
4. Show the evaluation table: grounded rate and residual hallucination rate.

Part 4C - On-device SLM fallback, router and queue

Timing: Weeks 6-8 (tag v0.4.3-4c)

Showcase for 4C

Turn on airplane mode and generate a short set of flashcards on the phone itself, checked by the same validator. Then simulate a rate limit (the server returns 429) and show the app switching to the local model without the student noticing anything except a small 'Made on this device' badge. On a low-RAM profile the job is queued and finishes automatically when the network returns.

Real at the end	Still a stand-in
llamadart runtime, model download manager, capability checks, `GenerationRouter` (cloud -> local -> queue), `PendingJobs` queue, debug failure-injection menu.	None. This part only adds resilience; the cloud path is unchanged.

WP4C.1 Model selection and benchmark

Lead: Zain (lead)

1. Pick two or three small instruction-tuned models in GGUF format, roughly 0.5 to 1.5 billion parameters at 4-bit quantisation (small enough for a 4 GB phone).
2. Benchmark on a real mid-range phone: load time, tokens per second, peak RAM, battery drop over 10 minutes, device temperature, and validator pass rate on 10 notes.
3. Pick one model and record the decision with numbers in `/docs/benchmarks`.

Output: Benchmark table and the chosen model.

WP4C.2 Model delivery and runtime

Lead: Zain (lead)

1. Do not bundle the model in the APK. Download it on first use: Wi-Fi only by default, resumable, SHA-256 verified, with a storage-space check and a Settings screen showing size, progress and a Delete button.
2. Integrate `llamadart` and run inference in a background isolate; load lazily and unload after a period of inactivity.
3. Limit input to a maximum number of lines and output to a small token cap; the fallback handles simple tasks only (up to 5 flashcards, short simplification), exactly as the proposal states.
4. Use a short prompt and parse the output as JSON with a tolerant repair step; send every result through the validator from 4B.

Output: Local generation that returns valid Lesson JSON for short notes.

WP4C.3 Capability gate and router

Lead: Zain (lead)

1. Capability check before any local run: RAM at least 4 GB, battery at least 20 percent or charging, not in thermal throttling, enough storage. Use `device_info_plus`, `battery_plus` and a small platform channel where needed.

2. `GenerationRouter` order: cloud first; on timeout, 5xx, 429 or no network, use the local model if the task is simple and the device is eligible; otherwise queue.
3. Show a small badge for where the lesson came from (cloud or device) and never block the student with technical wording.
4. Add a debug menu to simulate offline, 429, slow network and low RAM so every path can be demonstrated on demand.

Output: Router with a unit test for each path using fake services.

WP4C.4 Offline queue

Lead: Zain (lead)

1. Add a `PendingJobs` table (jobId, noteHash, status, attempts, nextAttemptAt, createdAt) with a migration; add it to the ER diagram as an extension.
2. Retry on connectivity change with `connectivity_plus` and, if needed, `workmanager`; use exponential backoff and a maximum attempt count.
3. Show a 'Waiting for connection' list with Cancel; send a local notification when the lesson is ready (optional).

Output: Queue that survives app restarts.

Definition of done for part 4C

- ☐ Airplane-mode generation works on a 4 GB phone
- ☐ A 429 from the server triggers local fallback with no error screen
- ☐ Low-RAM or low-battery path queues the job and shows it
- ☐ Queued job completes automatically after reconnecting
- ☐ Model can be downloaded, verified and deleted from Settings

Demo script for part 4C

1. Open Settings and show the downloaded model.
2. Enable airplane mode, scan a short note, show the local lesson with the badge.
3. Disable airplane mode, use the debug menu to simulate a rate limit, generate again: local fallback.
4. Use the debug menu to simulate low RAM: job queued; turn the network on and watch it complete.

Week-by-week plan

When	Focus
Weeks 1-2	Part 4A: Gemini + prompts + cache
Weeks 3-5	Part 4B: validator v1 in week 3, v2 and calibration in weeks 4-5
Weeks 6-8	Part 4C: benchmark, runtime, router, queue; guerrilla test #2 in week 8

Deliverables

- APKs `v0.4.1-4a`, `v0.4.2-4b`, `v0.4.3-4c`
- Prompt pack v1
- Calibrated `grounding_config.json` and evaluation table
- Model benchmark report
- Updated ER and sequence diagrams
- Monthly report (February)

Definition of done for the whole milestone

- ☐ Each of the three parts has its own tag, demo script and passing tests
- ☐ If 4C slips, 4A + 4B still make a complete, honest demo (and vice versa)

- ☐ No secrets in the APK; server still stores no note content
- ☐ Guerrilla test #2 notes written up

Demo script (combined)

1. Run the three sub-demos in order (about 2 minutes each).
2. Finish with one combined run: airplane mode scan -> local lesson -> verified badges.

Say this honestly in the demo (limitations)

Local model quality is lower than Gemini; that is why it only handles simple tasks and every output is validated. Say this before someone asks.

Risks and fallbacks

Risk	Mitigation / fallback
Embedding model or tokenizer port takes too long	Ship validator v1 (lexical + entity check) first; it already catches wrong numbers and names. Add Sem later.
Local model too slow or hot on mid-range phones	Smaller model, shorter output cap, background isolate, battery and thermal gates, and the queue as the last resort.
Gemini free tier limits or changed terms	Single provider as agreed with the supervisor; cache aggressively on device; local fallback; check the current data-use terms and state them truthfully in the in-app privacy note.

Milestone 5 - Accessibility Features & QA

When	February - March (about 6 weeks)
Release tag	v0.5.0-m5

Goal. Turn a working app into an app that feels designed for each learner, then test it hard. Two independent parts: 5A (read-aloud and profile experiences) and 5B (internal QA and evaluation). Either can be shown without the other.

What you can showcase at the end of this milestone

Same lesson, three experiences. ADHD: one short card at a time, progress bar, short sessions. Dyslexia: dyslexia-friendly font, wider spacing, cream background, read-aloud with word highlighting. Autism: literal 'simplified wording' option, calm palette, no motion, clear step indicators. Plus the QA dashboard: OCR accuracy, validator metrics, performance table and the bug list.

Real at the end	Still a stand-in
Text-to-Speech, full profile themes, adaptive layouts, simplify mode, complete internal test suite and evaluation report.	Cross-question comprehension check is optional (stretch). Simplify mode uses the same proxy and validator.

Work packages

WP5.1 Text-to-Speech read-aloud (5A)

Lead: Ujala (lead), Zain (assist)

1. Integrate `flutter_tts` using the device's own voices, so it works offline and costs nothing.
2. Read a card as: question, short pause, answer. Controls: play/pause, speed 0.5x to 1.5x, voice and language picker; remember settings per profile.
3. Word or sentence highlighting using the plugin's progress callback where the platform supports it; fall back to whole-card highlighting.
4. Handle audio focus and interruptions (phone call, headphones unplugged) and stop speech when leaving the screen.
5. Add a 'Read lesson' continuous mode and an auto-read toggle (on by default for the dyslexia profile).

Output: Read-aloud on every card and in lesson overview, tested on Android and iOS.

WP5.2 Profile experiences (5A)

Lead: Ujala (lead)

1. ADHD: single-idea cards, one card per screen, visible progress bar, 5-card micro-sessions with a 'keep going?' prompt, minimal clutter, immediate feedback.
2. Dyslexia: bundle a dyslexia-friendly typeface under a licence that allows it (check the font licence, for example OFL fonts such as OpenDyslexic, Lexend or Atkinson Hyperlegible), extra letter and word spacing, line height of at least 1.5, left-aligned text, no italics or underlining for emphasis, off-white background.
3. Autism: predictable layout and navigation, literal wording, calm low-saturation palette, no autoplay motion (respect the system reduce-motion setting), explicit 'Step 2 of 4' indicators, confirmation before destructive actions, sounds off by default.
4. Let students fine-tune: font size, spacing, palette, speech speed. Describe these as accessibility aids based on published guidance, not clinical treatment.

Output: Three tested experiences driven by one theme/config object.

WP5.3 Simplify wording mode (5A)

Lead: Zain (lead), Ujala (UI)

1. Add `POST /api/v1/text/simplify` to the proxy with its own prompt: rewrite in shorter, literal sentences without changing facts.
2. Run every simplified sentence through the grounding validator against the original card text; discard or keep the original if it fails.
3. Allow the local SLM to handle short simplification when offline (same router).
4. Add a 'Simplify' toggle on cards, remembered for the autism profile by default.

Output: Simplify toggle working online and offline.

WP5.4 Optional: comprehension check (5A stretch)

Lead: Ujala and Zain

1. Use the optional `verificationQuestion` field to ask the student a question whose answer is in the note, with a typed or multiple-choice response checked against the source lines.
2. Only start this if 5.1-5.3 are done by the end of week 3; it is not required by the proposal.

Output: Short 'Check your understanding' step after a session.

WP5.5 Frontend and accessibility QA (5B)

Lead: Ujala (lead)

1. Device and display matrix: small phone, normal phone, tablet; text scale 1.0, 1.5, 2.0; light and dark where supported.
2. Golden (screenshot) tests for the main screens in each profile theme.
3. Run a TalkBack (Android) and VoiceOver (iOS) pass; fix missing labels and focus order.
4. Contrast audit of every palette; keyboard/switch navigation spot check.

Output: UI test report with screenshots and fixes.

WP5.6 OCR, backend, database and performance QA (5B)

Lead: Zain (lead), Ujala (OCR)

1. OCR: rerun the 30-note dataset with the final pipeline and report CER and WER (Ujala).
2. Backend: xUnit coverage for validators, rate limiter and error mapping; load test with k6 or NBomber at an agreed concurrency (for example 20 virtual users) and report median and p95 latency.
3. Database: migration tests, a 1,000-card performance test, encryption verification, delete-all-data test.
4. On-device performance: latency, peak RAM, battery drop and temperature for the local model, and OCR + validator times, on at least two phones.
5. Validation: final calibration and the comparison table (Gemini, local SLM, commercial assistant): Grounded Rate and residual Hallucination Rate.
6. Triage all bugs as P0 (crash/data loss), P1 (main flow broken), P2 (cosmetic); fix P0 and P1 before tagging.

Output: QA report with tables and charts ready to paste into the final report.

Week-by-week plan

When	Focus
Weeks 1-2	5A: TTS and profile experiences
Week 3	5A: simplify mode; optional comprehension check
Weeks 4-5	5B: frontend, OCR, backend, database, performance QA
Week 6	Fix P0 and P1 bugs, rehearse, tag `v0.5.0-m5`

Deliverables

- APK `v0.5.0-m5`
- QA and evaluation report
- Performance table
- Bug list with status
- Midterm and monthly reports (March)

Definition of done

☐ Each profile has a distinct, documented experience
☐ Read-aloud works offline
☐ No overflow at 2.0x text scale; contrast audit passed
☐ All P0 and P1 bugs fixed; known P2 list written
☐ Final grounded-rate and hallucination-rate table exists

Demo script (about 3-5 minutes)

1. Open the same lesson with the ADHD profile, then dyslexia (with read-aloud), then autism (with simplify).
2. Show a failure case: network off, local fallback, queued job.
3. Open the QA report: accuracy, performance and validation tables.

> **Say this honestly in the demo (limitations)**
> These profile designs follow published accessibility guidance but are not clinically validated; formal feedback comes from UAT in Milestone 6.

Risks and fallbacks

Risk	Mitigation / fallback
TTS voices vary by device	Test on at least two Android brands and one iPhone; let the student choose a voice and fall back to the default.
QA finds big problems late	Run weekly bug triage from M3 onwards so M5 is a clean-up, not a discovery phase.

Milestone 6 - Finalisation & Documentation

When	March - April (about 6 weeks)
Release tag	v1.0.0

Goal. Prove the app works for real users, harden it for release, and finish the report, poster and presentation. Three independent parts: 6A UAT, 6B release hardening, 6C documents and presentation.

What you can showcase at the end of this milestone

The final release build, the evaluation results (OCR, validator, performance), UAT results with SUS scores, a 2-minute demo video, and the offline and cloud paths working live.

Real at the end	Still a stand-in
Everything. This milestone changes quality, not features.	Nothing is stood in for. Debug menus are removed or hidden behind a build flag.

Work packages

WP6.1 User Acceptance Testing (6A)

Lead: Ujala (lead), Zain (support)

1. Get supervisor and department approval for the study; prepare a one-page information sheet and consent form; anonymise all data and allow withdrawal at any time. If participants are under 18, parental consent is required.
2. Recruit 5-10 participants, including neurodivergent students if possible (for example through university student-support services); otherwise recruit peers and state this limitation honestly in the report.
3. Prepare a task script: install and choose a profile, scan a note, fix OCR text, generate, review five cards, use read-aloud, simplify one card, work in airplane mode, delete data.
4. Record per task: success or fail, time, number of errors, and observer notes. After the session give the System Usability Scale (10 questions) and one extra single-item rating of mental effort, because SUS measures perceived usability rather than cognitive load.
5. Compute SUS scores (average 68 is a common benchmark), summarise findings, run a one-week fix sprint on the top issues, and retest with 2-3 participants.

Output: UAT report with tables, quotes (anonymous) and a list of changes made.

WP6.2 Full system integration testing (6B)

Lead: Zain (lead)

1. Write end-to-end tests for the main journey (`integration_test`) and a manual script for camera steps.
2. Test the scenario matrix: online or offline x limit reached or not x RAM sufficient or not x each profile.
3. Run a stability test of 50 consecutive scans without restart; watch memory and temperature.
4. Test on real Android and iOS devices; if no Mac is available for iOS, use a friend's Mac or a cloud build service and state this in the limitations.

Output: Integration test report with pass/fail matrix.

WP6.3 Release hardening (6B)

Lead: Zain (lead), Ujala (polish)

1. Remove or hide debug menus and mock generator in release flavour; verify there are no secrets in the APK and no debug logging of note content.

2. Add a global error handler (`runZonedGuarded`), friendly error pages, empty states, and a first-run onboarding that explains what is stored where.
3. Write an honest in-app privacy page: the note text is sent to our server and on to the Gemini API only when generating in the cloud; the server stores no content; all lessons stay on the device. Check Google's current Gemini API data-use terms before wording this.
4. App icon, splash, name, screenshots and release notes; signed release APK/AAB; deploy the final server and rotate every key used during development.
5. Code freeze one week before the final presentation; only P0 fixes after that.

Output: Signed release build and `v1.0.0` GitHub Release.

WP6.4 Final report and documentation (6C)

Lead: Ujala (lead), Zain (technical chapters)

1. Outline: Introduction, Literature Review, Requirements, Design (architecture, DB, validator), Implementation, Testing and Evaluation, UAT, Limitations, Conclusion and Future Work.
2. Paste tables and figures from earlier milestones: OCR baseline (M3), validator and hallucination rates (M4B), model benchmark (M4C), QA and performance (M5), UAT and SUS (M6).
3. Write the user manual, developer README (setup, run, test, deploy), API documentation and the deployment guide.
4. Run Turnitin and AI-report checks early enough to rewrite flagged parts, and collect the signed appendices.

Output: Final report, user manual and a clean README.

WP6.5 Poster, slides and demo (6C)

Lead: Ujala (lead), Zain (demo)

1. Update the master presentation with real screenshots and measured results; keep a fixed split of who presents what.
2. Design the poster: problem, solution, architecture, key results, QR code to the repository.
3. Record a 2-minute demo video and keep the final APK on the phone and as a backup file.
4. Rehearse three times, including a run with Wi-Fi off, and prepare a one-page question-and-answer sheet (why hybrid, why grounding score, what it does not guarantee, privacy).

Output: Poster, slides, video and rehearsed demo.

Week-by-week plan

When	Focus
Weeks 1-2	6A UAT preparation, sessions and analysis; 6B integration tests start
Week 3	UAT fix sprint and retest; release hardening
Weeks 4-5	6C report, README, poster, slides, demo video
Week 6	Freeze, final checks, rehearsals, tag `v1.0.0`

Deliverables

- Signed release APK `v1.0.0` and live proxy
- UAT report with SUS
- Integration test report
- Final report, user manual, README
- Poster, slides, demo video
- Monthly reports (April) and final FYP report

Definition of done

- ☐ Every proposal objective in Section 1.3 is mapped to evidence
- ☐ UAT done with SUS scores and a before/after change list
- ☐ No P0 or P1 bugs open; known issues documented
- ☐ Release build contains no debug tools or secrets
- ☐ Report, poster and slides complete and rehearsed

Demo script (about 3-5 minutes)

1. Run the 7-minute final demo: scan -> verified lesson -> read-aloud -> airplane mode fallback -> progress.
2. Show the results slide: OCR, grounded rate, hallucination rate, performance, SUS.
3. Answer the prepared hard questions.

Say this honestly in the demo (limitations)

State clearly what the validator does not guarantee (it checks consistency with the notes, not truth) and which parts of UAT were limited by participant numbers.

Risks and fallbacks

Risk	Mitigation / fallback
Participants hard to recruit	Start recruitment in M5; offer short 20-minute sessions; accept peers with clear labelling.
Report written too late	Write chapters at the end of each milestone from week 1 of M1 onward; M6 only polishes.

Cross-cutting: risks and demo-day checklist

Project risk register

Risk	Level	Mitigation
OCR accuracy on messy handwriting	High	Guided capture, preprocessing A/B test, editable review screen, week-1 go/no-go gate, contingency vision OCR through proxy.
Gemini limits, outage or policy change	High	Single provider, on-device cache, local fallback, queue, mock mode for demos, current terms checked and stated truthfully.
Local model too heavy for phones	Medium	Small quantised model, capability gate, token caps, background isolate, queue as last resort.
Embedding model / tokenizer port slips	Medium	Validator v1 (lexical + entity) ships first; semantic score added after.
Free-tier server cold starts	Medium	Warm-up step, laptop-hotspot dev flavour as backup, pre-recorded demo video.
Team schedule clashes with exams	Medium	Buffer weeks at the end of M2 and M3; each milestone's last week is slack.
Scope creep (extra features)	Medium	Could-have list; stretch items (comprehension check) only after Must items pass.
UAT recruitment and ethics	Medium	Start early, get approvals, accept peers with honest limitation statement.
iOS testing without a Mac	Low-Medium	Borrow a Mac or use a cloud build service; state the limitation if needed.
Secrets leaked in the repository	High	`.gitignore` rules, secret scanning, keys only in server environment, rotate before release.

Demo-day checklist (use at the end of every milestone)

- ☐ `main` builds; CI green; tag created; signed APK attached to the GitHub Release
- ☐ Demo phone charged, notifications silenced, screen timeout extended, correct build flavour installed
- ☐ Backend warmed up (`/health`) and `Generator:Mode` set as planned; laptop hotspot ready as backup
- ☐ Seed data and 2-3 prepared sample photos on the phone
- ☐ Feature flags checked: only finished features are visible
- ☐ 90-second screen recording saved on phone and laptop
- ☐ Limitations slide ready (what is real, what is stand-in)
- ☐ One person drives, one narrates; rehearsed with Wi-Fi off at least once

Appendix A - Database tables (on-device and backend)

Tables 1-5 are exactly the proposal's Section 5.3. The suggested extra columns and the `PendingJobs` table are small extensions; if you adopt them, update the ER diagram (Figure 5.2) in the report.

Table	Columns	Note
UserProfiles	ProfileID (PK), ProfileType (ADHD | Dyslexia | Autism), ThemeSettings	As proposal. Optionally add TextScale and TtsSettings inside ThemeSettings JSON.
SourceNotes	NoteID (PK), RawOCRText, DocumentHash, CreatedAt	As proposal. DocumentHash is SHA-256 of normalised text.
Lessons	LessonID (PK), NoteID (FK), Title, ConceptMapJSON	As proposal. Suggested extra columns: PromptVersion, Source (seed | mock | cloud | device).
Flashcards	CardID (PK), LessonID (FK), Question, Answer, SourceLineReference	As proposal. Suggested extra columns: GroundingScore, Verified (bool).
ReviewProgress	ProgressID (PK), CardID (FK), SpacedRepetitionInterval, NextReviewDate	As proposal. Leitner scheduling needs no more columns.
PendingJobs (extension)	JobID (PK), NoteHash, Status, Attempts, NextAttemptAt, CreatedAt	New in M4C for the offline queue. Add it to the ER diagram.
Backend: RequestLog	Id (PK), InstallHash, Endpoint, Status, LatencyMs, Provider, CreatedAt	Metadata only; no note content (Figure 5.3).

Appendix B - Lesson JSON contract (freeze in M1)

Every generator (seed, mock, Gemini, on-device model) must return this shape. `sourceLines` are the 1-based OCR line numbers shown on the review screen, which is what makes the grounding check and the 'Where is this from?' sheet possible.

```
{
  "schemaVersion": 1,
  "title": "string",
  "meta": { "profile": "ADHD|Dyslexia|Autism", "source": "seed|mock|cloud|device",
            "promptVersion": "v1" },
  "conceptMap": {
    "nodes": [ { "id": "n1", "label": "string", "sourceLines": [3, 4] } ],
    "edges": [ { "from": "n1", "to": "n2", "label": "string" } ]
  },
  "flashcards": [
    { "id": "c1", "question": "string", "answer": "string",
      "sourceLines": [3], "chunks": ["optional short pieces"] }
  ],
  "verificationQuestion": { "question": "optional", "sourceLines": [5] }
}
```

Appendix C - Grounding score cheat sheet (proposal Section 5.1)

Step	What to implement
1. Normalise	Lowercase, strip punctuation and OCR hyphenation, remove stop words; evidence E = cited lines plus one neighbour each side.
2. Lexical	Cn = |Nn(g) intersect Nn(E)| / |Nn(g)|; L = (C1 + C2) / 2.
3. Semantic	Sem = max over evidence lines of cosine(embedding(g), embedding(line)).
4. Combine	G = 0.6 * Sem + 0.4 * L. Entity check X: every number, date, formula and proper name in g must appear in E exactly; if X = 0 then G = 0.
5. Decide	X = 1 and G >= 0.75: accept. 0.50 <= G < 0.75: regenerate once with a quote-only prompt. Otherwise discard. Thresholds and alpha are calibrated by grid search on labelled data.

What the score does and does not do

It checks that an item is supported by the student's own notes. It cannot tell whether the notes themselves are correct. Report the measured residual hallucination rate; do not claim zero.

Appendix D - Suggested monorepo layout

```
repo-root/
  app/                      Flutter app (feature-first)
    lib/
      core/                 config (AppConfig), theme, routing, errors, utils
      data/                 drift database, DAOs, repositories
      features/
        onboarding/  library/  lesson/  review/  concept_map/
        capture/  ocr/  generation/  validation/  fallback/
        tts/  settings/  progress/
      services/             interfaces + implementations (seed, remote, local)
    assets/seed/            seed lessons (M2)
    assets/models/          validator config, stop words, vocab file
    test/  integration_test/
  backend/
    src/Api/  src/Core/  src/Infrastructure/  prompts/v1/
    tests/
  docs/                     proposal, contracts, diagrams, testdata, benchmarks
  design/                   exported Figma assets
  .github/workflows/        ci-app.yml, ci-backend.yml, deploy.yml
```

Appendix E - Packages and tools by milestone

Package names are the usual ones at the time of writing. Check pub.dev and NuGet for the current maintained versions before adding each one, and prefer the smallest dependency that does the job.

Milestone	Main packages and tools
M1	flutter_riverpod, go_router, flutter_lints; ASP.NET Core (not yet), GitHub Actions
M2	drift, drift_dev, build_runner, sqlcipher_flutter_libs, flutter_secure_storage, crypto, graphview (or CustomPainter), mocktail
M3	camera, permission_handler, image_picker (debug), opencv_dart or image, google_mlkit_text_recognition, dio, path_provider; backend: ASP.NET Core minimal API, EF Core SQLite, Serilog, xUnit, Microsoft.AspNetCore.Mvc.Testing
M4	ONNX Runtime or TFLite Flutter binding, llamadart, device_info_plus, battery_plus, connectivity_plus, workmanager (optional); backend: Microsoft.Extensions.Http.Resilience, Gemini REST API
M5	flutter_tts, a licensed dyslexia-friendly font, integration_test, golden tests; k6 or NBomber for load testing
M6	integration_test, runZonedGuarded, release signing, GitHub Releases